

COMPOSE 2 Y 3 --SOLO ALUMNOS AUSENTES

[Diapositiva 118 ↗](#)

[Diapositiva 122 ↗](#)

Instrucciones

Realiza las dos prácticas de Docker Compose indicadas a continuación. En cada una debes poder construir y arrancar la aplicación mediante Compose y documentar cómo comprobar su funcionamiento.

EJERCICIO 2

Flask Cat App con Compose

[Ver diapositiva 118](#)

Dockeriza con Compose Flask Cat App, la aplicación de Python y Flask que muestra un GIF aleatorio de un gato al recargar la página.

Parte del [ejercicio original de Flask Cat App](#) y del archivo `templates.zip` enlazado en los materiales. Si ya has realizado la práctica, utiliza tu proyecto como base.

Preparar la aplicación

- Lee `app.py` y la plantilla para deducir la carpeta donde deben colocarse los archivos `.gif`. Añade al menos seis GIFs de gatos en la ubicación correcta.
- Prepara un `Dockerfile` con una imagen de Python 3 sobre Alpine, instalación de `requirements.txt` sin caché y copia del código y los recursos.
- La aplicación debe ejecutarse con un usuario que no sea `root`, exponer el puerto 5000 y arrancar con `python app.py`.

Integrar Docker Compose

- Define el servicio de la aplicación y su construcción a partir del `Dockerfile`. Conserva el nombre de imagen del ejercicio original: `flask-cat-app:alumnoXX`, sustituyendo `XX` por tu identificador.
- Configura el acceso desde el puerto 8888 del equipo al puerto 5000 del contenedor.
- Construye y arranca la aplicación con Compose. Abre `http://localhost:8888/` y recarga varias veces para comprobar la selección aleatoria de los GIFs.

Documenta los comandos de Compose, incluye una captura de la aplicación funcionando y explica cómo dedujiste la ubicación de los GIFs.

EJERCICIO 3

Dice App con Compose y variables de entorno

[Ver diapositiva 122](#)

Dockeriza con Compose Dice App. Utiliza el proyecto disponible en el enlace de Google Drive de los materiales de esta tarea.

- Prepara los archivos necesarios para construir y arrancar la aplicación con Docker Compose.
- Utiliza un archivo `.env` para poder cambiar el número de caras de los dados. Asegúrate de que la configuración llega a la aplicación.
- Si el archivo `.env` no existe o está vacío, la aplicación debe arrancar y utilizar **seis caras por defecto**.

Comprobaciones

1. **Sin archivo `.env`:** los dados tienen seis caras.
2. **Con `.env` vacío:** se mantiene el valor predeterminado de seis caras.
3. **Con un número de caras configurado:** la aplicación utiliza ese valor.

Incluye los pasos y las capturas que demuestren el funcionamiento en los tres casos.

Entrega de las dos prácticas

Adjunta el enlace o los enlaces a los repositorios de GitHub. Identifica claramente cada ejercicio e incluye su código, sus archivos de Docker y Compose y un `README.md` con los comandos de construcción, arranque y parada, las comprobaciones solicitadas y sus capturas.

Materiales de la tarea

Los archivos adjuntos se descargan por separado desde la página de esta tarea.



templates.zip · Flask Cat App

Material del ejercicio 2 · mismo archivo que la tarea original



Material de Dice App en Google Drive

Proyecto del ejercicio 3 · abrir en Google Drive

